

FIFO Analysis Toolkit — Quick Start Guide v1.2

Audience: Maintenance Engineers and Test Engineers | **Goal:** Get you running your first analysis in under 10 minutes.

1 Installation

Requirements: Python 3.9+ and four packages.

```
pip install numpy scipy matplotlib
```

(`csv`, `json`, `argparse`, `math` are built into Python — no install needed.)

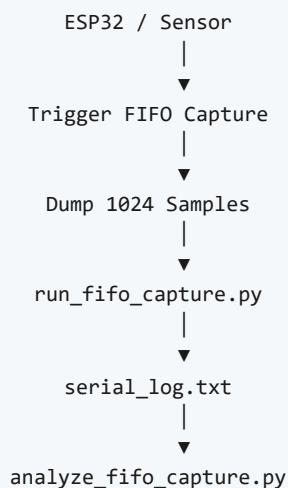
Verify it's ready:

```
python analyze_fifo_capture.py --help
```

If you see a usage message with `--sr-hz`, `--manufacturer`, `--bearing`, `--shaft-rpm`, you're ready to go.

2 Capturing FIFO Data

Before you can analyze anything, you need a capture file. The normal workflow uses the capture tool — **manually copy-pasting data out of the Serial Monitor is not the normal workflow** and is easy to get wrong (truncated rows, missing header/footer lines).



Important: `run_fifo_capture.py` captures the serial stream — it does NOT generate FIFO data. The FIFO capture itself (the 1024-sample dump) happens on the ESP32/sensor firmware; `run_fifo_capture.py` only sends the trigger command over serial and listens for the result. If the device isn't connected or doesn't produce a capture, the tool has nothing to record.

What happens, step by step:

1. **Connect** the ESP32 / sensor over USB/serial.
2. **Run the capture tool** (`python run_fifo_capture.py`). It opens the serial port and sends the FIFO trigger command for you.
3. **The tool waits** for the FIFO capture header line before it starts collecting data — it will not save a partial or misaligned capture.
4. **All 1024 samples are received** (dumped by the device) and appended to the log file (`serial_log.txt`), terminated by the `FIFO_CAPTURE_END` marker.
5. **The CSV is saved automatically** — you don't hand-edit or reformat anything.
6. **The tool then runs the analyzer for you**, calling `analyze_fifo_capture.py serial_log.txt --sr-hz 1000` automatically.

You can also point `analyze_fifo_capture.py` at a CSV you received some other way (e.g. a Phase 7A-format sample file) — the capture tool above is the *recommended* path, not the only supported input source.

3 Folder Structure

The toolkit supports two input types side by side — a legacy capture log and a Phase 7A sample CSV — so both can appear in the same folder:

```
experimental/
  analyze_fifo_capture.py
  run_fifo_capture.py
  serial_log.txt
  phase7a_sample_captureId1.csv
  bearing_database/
```

Item	Purpose
<code>analyze_fifo_capture.py</code>	Main tool — reads a capture file, produces all plots + summary CSV.
<code>run_fifo_capture.py</code>	Captures the serial stream from the device (Section 2) and then runs the analyzer for you. Does not generate FIFO data itself.
<code>serial_log.txt</code>	Legacy-format input — captures accumulate here, appended to each run.
<code>phase7a_sample_captureId1.csv</code>	Phase 7A-format input — a standalone CSV, supplied directly (no capture tool needed for this format).
<code>bearing_database/</code>	Contains <code>bearing_db.json</code> — bearing geometry (manufacturer → model → dimensions) used by <code>--manufacturer</code> / <code>--bearing</code> .

Both `serial_log.txt` and any Phase 7A `.csv` file are valid inputs to `analyze_fifo_capture.py` — the format is auto-detected, no flag needed.

4 Before Running Analysis — Checklist

Quick sanity check before you trust the results:

- ☐ CSV/log contains a full 1024-sample capture (not cut off mid-stream)
- ☐ Sample rate is known (for `--sr-hz`)
- ☐ Bearing model is correct, if using the overlay (matches the nameplate)
- ☐ Shaft RPM is measured, not guessed, if using the overlay
- ☐ Capture file is complete (ends with `FIFO_CAPTURE_END` , or is a full Phase 7A CSV)

If any box is unchecked, see **Section 9 — Troubleshooting**.

5 Running the Tool

Minimum command — just a capture file:

```
python analyze_fifo_capture.py serial_log.txt
```

Almost always you also want `--sr-hz` so frequency axes are labeled in real Hz instead of "cycles per block":

```
python analyze_fifo_capture.py serial_log.txt --sr-hz 1000
```

The tool auto-detects whether your file is the legacy 9-column format or the Phase 7A 4-column format — no flag needed for that.

Every run prints progress to the console and writes a set of PNG plots plus one `*_summary.csv` next to your working directory (or wherever `--out-prefix` points).

6 Reading the Results

File	What it shows	Look for...
<code>*_timeseries.png</code> (Waveform)	Raw waveform, X/Y/Z vs. time/sample	Obvious clipping, dropouts, or gross amplitude changes at a glance.
<code>*_fft.png</code> (FFT)	Frequency spectrum of the whole capture	Dominant vibration frequencies (running speed, gear mesh, etc.).
<code>*_spectrogram.png</code> (Spectrogram)	Frequency content over time (color = magnitude)	Transient or intermittent events an FFT alone would average away.
<code>*_envelope.png</code> (Envelope)	Amplitude envelope of the waveform (Hilbert transform)	Impacting/modulation patterns — the "shape" of repetitive impacts.
<code>*_envelope_fft.png</code> (Envelope FFT)	Frequency spectrum of that envelope	The repetition rate of impacts — where bearing fault frequencies show up.
Bearing Overlay (on Envelope FFT)	Dashed lines at theoretical shaft_hz/FTF/BPFO/BPFI/BSF	Whether a real peak lines up with a line — evidence to investigate, not a diagnosis.

Rule of thumb: start with the waveform and FFT for a quick sanity check, then go to the envelope FFT (with the bearing overlay, if applicable) when you're specifically hunting for bearing fault signatures.

7 Typical Workflow

```

Capture FIFO on device
    |
    v
  Save as CSV
    |
    v
Run analyze_fifo_capture.py --sr-hz <rate>
    |
    v
Check Waveform -> anything clipped or dead?
    |
    v
Check FFT      -> what frequencies dominate?
    |
    v
Check Spectrogram -> steady or transient?
    |
    v
Check Envelope FFT -> repetitive impact rate?
    |
    v
Compare against Bearing Frequencies (optional, Section 8)
    |
    v
Review *_summary.csv for exact numbers

```

8 Bearing Overlay

If you know the bearing model and shaft speed, the tool can mark the bearing's theoretical fault frequencies directly on the envelope FFT plot.

```
python analyze_fifo_capture.py serial_log.txt --sr-hz 1000 \
  --manufacturer SKF --bearing 6205 --shaft-rpm 1800
```

This draws five dashed lines on `*_envelope_fft.png`:

Line	Meaning
<code>shaft_hz</code>	Shaft rotation rate
<code>FTF</code>	Cage (train) frequency
<code>BPFO</code>	Ball Pass Frequency — Outer race
<code>BPFI</code>	Ball Pass Frequency — Inner race
<code>BSF</code>	Ball Spin Frequency

How to read it: if a real spectral peak lines up closely with one of these dashed lines, that's evidence worth investigating on that element (outer race, inner race, cage, or ball). It is *not* an automatic diagnosis — the tool draws the lines, you make the judgment call.

Notes:

- `--manufacturer` and `--bearing` must be given together.
- `--shaft-rpm` must be a real, known value — the tool never estimates it. No RPM means no lines are drawn (and the CSV frequency columns stay blank).
- Only frequencies below Nyquist (`sample_rate_hz / 2`) are drawn.

9 Troubleshooting

Symptom	Likely Cause	Fix
No FIFO captures found in this file	File doesn't match either supported header format	Check the file starts with <code>FIFOIndex,Tag,SR,...</code> or <code>Index,X,Y,Z</code> , and (for legacy) has a <code>FIFO_CAPTURE_END</code> line.
CSV incomplete / fewer than 1024 samples	Capture was interrupted, or the serial log was cut off mid-stream	Check <code>sample_count</code> in the summary CSV; re-run the capture and confirm it reaches <code>FIFO_CAPTURE_END</code> .
Wrong sample rate	<code>--sr-hz</code> doesn't match the sensor's actual configured rate	Confirm the real sample rate and re-run with the correct <code>--sr-hz</code> . Symptoms: implausible frequency numbers, overlay lines that don't line up with real peaks.
Wrong bearing model	<code>--manufacturer</code> / <code>--bearing</code> point at a valid entry, but not the bearing actually installed	Double-check the physical bearing's nameplate/designation against <code>bearing_db.json</code> .
Missing RPM / no bearing lines drawn	<code>--shaft-rpm</code> missing, zero, or negative	Supply a real, positive, measured shaft RPM — the tool never estimates it.
Unknown manufacturer '...'	Manufacturer not in <code>bearing_database/bearing_db.json</code>	Check spelling/case, or add the manufacturer to the JSON file.
Unknown bearing model '...'	Bearing model not listed under that manufacturer	Check the model string, or add it to the JSON file.
Frequency axis says "cycles per block"	<code>--sr-hz</code> not supplied	Add <code>--sr-hz <your sample rate></code> .
Bearing database is not valid JSON	<code>bearing_db.json</code> has a syntax error	Validate the JSON and fix.
Bearing database malformed: ... missing one of [...]	A bearing entry is missing a required field	Add <code>rolling_elements</code> , <code>ball_diameter_mm</code> , <code>pitch_diameter_mm</code> , <code>contact_angle_deg</code> .

10 Command Cheat Sheet

Typical Capture

```
# Connect the sensor, then capture + auto-analyze in one step
python run_fifo_capture.py
```

Typical Analysis

```
# Basic run
python analyze_fifo_capture.py serial_log.txt

# With real sample rate (recommended, always)
python analyze_fifo_capture.py serial_log.txt --sr-hz 1000

# Phase 7A input (auto-detected, same command shape)
python analyze_fifo_capture.py phase7a_sample.csv --sr-hz 1000

# Custom output name/location
python analyze_fifo_capture.py serial_log.txt --sr-hz 1000 --out-prefix run042
```

Bearing Analysis

```
# Bearing geometry lookup only (no frequency numbers yet)
python analyze_fifo_capture.py serial_log.txt --manufacturer SKF --bearing 6205

# Full pipeline: geometry + frequencies + overlay
python analyze_fifo_capture.py serial_log.txt --sr-hz 1000 \
    --manufacturer SKF --bearing 6205 --shaft-rpm 1800

# See all options
python analyze_fifo_capture.py --help
```

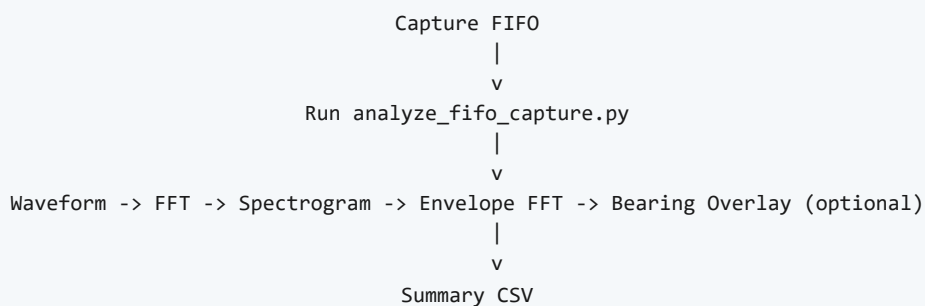
Output Files

```
*_timeseries.png    Waveform
*_fft.png           FFT
*_spectrogram.png   Spectrogram
*_envelope.png       Envelope
*_envelope_fft.png   Envelope FFT (+ bearing overlay, if enabled)
*_summary.csv        One row per capture, all numeric results
```

Option	Required With	Purpose
<code>--sr-hz</code>	—	Real sample rate in Hz (frequency axes, overlay).
<code>--out-prefix</code>	—	Output filename prefix (default <code>fifo_analysis</code>).
<code>--manufacturer</code>	<code>--bearing</code>	Bearing manufacturer (e.g. <code>SKF</code>).
<code>--bearing</code>	<code>--manufacturer</code>	Bearing model (e.g. <code>6205</code>).
<code>--shaft-rpm</code>	bearing selected	Shaft speed in RPM for frequency calc + overlay.

11 Quick Validation Example

A fast end-to-end check that everything works:



```
python run_fifo_capture.py
# ...or, if you already have a capture file:
python analyze_fifo_capture.py serial_log.txt --sr-hz 1000
```

Confirm:

- ☐ Waveform generated
- ☐ FFT generated
- ☐ Spectrogram generated
- ☐ Envelope FFT generated
- ☐ Summary CSV generated

If every item succeeds, the FIFO Analysis Toolkit is ready for use.

For full column-by-column CSV documentation, JSON schema details, known limitations, and validation results, see [FIFO_Analysis_Toolkit_User_Guide_v1.0](#) (full reference) and [fifo-analysis-v1.0](#) (release notes).